

5. Prometheus whit. PromQL

1. Kubernetes 환경에서의 Prometheus 모니터링 설정

Kubernetes 클러스터에 Prometheus 모니터링 시스템을 효율적으로 구축하는 방법을 Helm과 Prometheus Operator를 활용하여 알아본다.

1.1. Prometheus 모니터링 스택의 구성 요소

1. Prometheus 모니터링의 핵심 구성 요소

- a. Prometheus 서버는 메트릭 데이터를 저장하고 처리하는 역할을 한다.
- b. 알림 관리자는 Prometheus가 수집한 데이터를 기반으로 알림을 보낸다.
- c. Grafana와 같은 데이터 시각화 도구를 통해 Prometheus 메트릭 데이터를 시각화할 수 있다.

2. Kubernetes에 Prometheus를 배포하는 방법

- a. **수동 설정:** 각 구성 요소에 필요한 모든 설정 파일을 직접 준비하는 방법이다.
 - i. 이 방법은 비효율적이고 많은 노력이 필요하며, 종속성 때문에 올바른 순서로 실행해야 하는 어려움이 있다.
- b. **Operator 활용:** 사용자가 생성하는 Prometheus 개별 구성 요소를 관리하는 방식으로, StatefulSet, Deployment 등을 하나의 단위로 관리하여 수동 관리가 필요 없도록 한다.
 - i. Operator는 Pod가 다운되면 재시작하고 항상 접근 가능한 상태를 유지하는 등의 작업을 수행한다.
- c. **Helm 차트 사용:** Operator를 배포하는 가장 효율적인 방법으로, Prometheus Operator는 Helm 커뮤니티에서 관리하는 Helm 차트를 제공한다.
 - i. 이 비디오에서는 Helm 차트를 사용하여 Prometheus 모니터링을 설정한다.

1.2. Helm을 이용한 Prometheus 모니터링 스택 배포

1. Helm 차트 설치 및 구성 요소 확인

- a. 미니큐브 환경에서 Prometheus Operator Helm 차트를 설치한다.
- b. `helm install prometheus prometheus-community/prometheus` 명령어를 사용하여 차트를 배포한다.

- c. 배포 후 몇 분 안에 모든 구성 요소가 시작되며, `kubectl get all` 명령어로 생성된 Kubernetes 구성 요소를 확인할 수 있다.
- d. **주요 구성 요소:**
 - i. **StatefulSet:** Prometheus 서버와 Alert Manager가 StatefulSet으로 배포된다.
 - 1. Prometheus 서버는 메인 이미지를 기반으로 하며, Operator에 의해 관리된다.
 - 2. Alert Manager도 세 개의 인스턴스로 구성된다.
 - ii. **Deployment:** Prometheus Operator, Grafana, CubeStateMetrics가 Deployment로 배포된다.
 - 1. Prometheus Operator는 Prometheus와 Alert Manager의 StatefulSet을 생성하는 핵심 역할을 한다.
 - 2. Grafana는 자체 배포로 구성된다.
 - iii. **DaemonSet:** Node Exporter가 DaemonSet으로 모든 워커 노드에서 실행된다.
 - 1. Node Exporter는 서버 자체의 메트릭을 Prometheus 메트릭으로 변환하여 스크래핑할 수 있도록 한다.
 - iv. **ReplicaSet:** 배포에 의해 생성되며, Grafana, CubeStateMetrics, Prometheus Operator의 기본 구성 요소이다.
- e. **서비스:** 각 구성 요소에는 자체 서비스가 있으며, Alert Manager, Grafana 등이 포함된다.

2. 기본 설정 및 CRDS (Custom Resource Definitions)

- a. 생성된 ConfigMap과 Secret에는 Prometheus 모니터링 스택의 기본 설정이 포함되어 있다.
 - i. Prometheus가 메트릭을 수집하는 방법, 기본 규칙 파일, 인증서, 사용자 이름 및 암호 등이 포함된다.
- b. 이러한 설정은 Operator에 의해 관리되며, 사용자가 직접 YAML 파일을 작성하는 것보다 훨씬 간편하다.
- c. CRDS는 Kubernetes의 또 다른 개념으로, Prometheus 모니터링 스택 설정에 포함되어 있다.

3. 각 구성 요소의 상세 내용 확인

a. Prometheus 서버:

- i. `describe statefulset prometheus` 명령어로 Prometheus StatefulSet을 확인할 수 있다.
- ii. Prometheus Pod에는 Prometheus 서버, 구성 리로더, 규칙 구성 리로더 컨테이너가 포함된다.
- iii. Prometheus 컨테이너는 특정 이미지와 버전을 사용하며, 노출된 포트와 인수, 마운트 정보를 확인할 수 있다.
- iv. Prometheus 구성 파일은 스크래핑해야 하는 엔드포인트를 정의하며, 규칙 파일은 알림 규칙 등을 정의한다.
- v. 구성 리로더는 구성 변경 시 Prometheus에 알리는 역할을 한다.
- vi. Prometheus 설정 파일과 규칙 파일은 ConfigMap으로 생성되며, Pod 볼륨에 마운트된다.
- vii. Prometheus 설정 파일은 Secret으로 관리될 수도 있다.

b. Alert Manager:

- i. Alert Manager의 메인 컨테이너에는 기본 설정 파일(`alertmanager.yaml`)과 설정 재로드 기능이 있다.

c. Prometheus Operator:

- i. Operator는 전체 Prometheus 모니터링 스택의 오케스트레이터 역할을 하며, 다양한 구성 요소를 관리하고 설정을 조율한다.

2. Prometheus Query Language (PromQL)와 데이터 시각화

PromQL은 Prometheus에서 수집된 시계열 데이터를 분석하고 시각화하는 데 사용되는 강력한 쿼리 언어이며, 이를 통해 시스템의 상태를 파악하고 문제를 진단할 수 있다.

2.1. PromQL의 탄생 배경과 특징

1. PromQL의 기원 및 개발

- a. Julius Volz는 Prometheus의 공동 창립자이자 PromQL의 초기 개발자이다.
- b. PromQL은 Google의 Borg 모니터링 시스템의 쿼리 언어에서 영감을 받아 개발되었다.

2. PromQL의 역할과 목적

- a. PromQL은 Prometheus에서 수집된 시계열 데이터를 분석하기 위한 언어이다.
- b. 데이터를 슬라이스하고 다이스하며, 변환, 집계, 비율 계산 등 다양한 연산을 수행할 수 있다.
- c. 이를 통해 인프라 및 서비스에서 발생하는 문제, 요청 수, 온도 등 다양한 정보를 얻을 수 있다.

3. PromQL의 주요 특징

- a. **시계열 데이터에 특화**: 시간 순서대로 기록된 데이터를 효율적으로 처리하도록 설계되었다.
- b. **벡터 매칭**: 두 시계열 집합 간의 이진 연산, 필터링 등 다양한 연산을 자동으로 수행한다.
 - i. 예를 들어, 수천 개의 호스트별 디스크 사용량과 총 디스크 크기를 비교하여 사용 비율을 계산할 수 있다.
- c. **SQL과의 차이점**: SQL과 같은 전통적인 쿼리 언어와 달리, 시계열 데이터 처리에 최적화되어 있다.
 - i. 벡터 매칭 기능은 PromQL의 혁신적인 기능 중 하나이다.

2.2. PromQL을 활용한 일반적인 쿼리 및 활용 사례

1. 일반적인 시스템 모니터링 쿼리

- a. **카운터 메트릭**: 서비스 요청 수와 같은 이벤트 발생 횟수를 나타내는 메트릭에 `rate` 함수를 사용하여 초당 증가율을 계산한다.
- b. **히스토그램**: 서비스 응답 시간 분포를 나타내는 히스토그램에서 90, 95, 99 백분위수 등을 계산한다.
- c. **비율 및 합계 계산**: 디스크 사용량과 총 디스크 크기의 비율과 같이 여러 메트릭 간의 비율이나 합계를 계산한다.
- d. **예측 쿼리**: 과거 사용량 데이터를 기반으로 디스크가 언제 채워질지 예측하는 등의 쿼리를 수행한다.

2. 다양한 활용 사례

- a. 대규모 데이터센터 IT 서비스 및 수천 개의 마이크로서비스 모니터링에 사용된다.
- b. 가정에서 온도, 습도, 압력 등을 모니터링하는 데 활용된다.
- c. 풍력 발전기의 전력 생산량 및 터빈 회전 각도를 모니터링하는 데 사용된다.

3. PromQL의 한계점

- a. Prometheus는 메트릭 데이터만 저장하며, PromQL은 메트릭 데이터에 대해서만 작동한다.
- b. 로그, 트레이스 등 다른 종류의 신호 유형은 다른 시스템과 언어를 사용해야 한다.

3. Prometheus 모니터링 스택의 시각화 및 접근

3.1. Grafana를 이용한 데이터 시각화

1. Grafana 접근 방법

- a. Prometheus 스택의 서비스들은 클러스터 IP 타입으로 내부 서비스이므로 외부에서 직접 접근할 수 없다.
- b. 프로덕션 환경에서는 일반적으로 Ingress를 구성하여 접근한다.
- c. 이 설정에서는 `kubectl port-forward` 명령어를 사용하여 Grafana에 접근한다.
- d. `kubectl port-forward deployment prometheus-grafana 3000:3000` 명령어를 실행하여 로컬 3000번 포트로 Grafana를 포워딩한다.
- e. 기본 사용자 이름은 `admin`이며, 비밀번호는 Prometheus Operator 차트에서 확인할 수 있다.

2. Grafana UI의 주요 기능

- a. **대시보드**: 전체 스택을 한눈에 볼 수 있으며, 다양한 알림 규칙을 확인할 수 있다.
- b. **PromQL 쿼리**: Prometheus 데이터베이스에서 데이터를 가져오기 위해 PromQL 쿼리 언어를 사용할 수 있다.
- c. **모니터링 항목**: Node Exporter는 노드 자체의 CPU, 메모리 사용량 등 메트릭을 스크래핑한다.
- d. **CubeStateMetrics**: Kubernetes 구성 요소(API 서버, 파드, 네임스페이스 등)의 CPU 사용량, 메모리 사용량, 대역폭 등의 메트릭을 수집한다.
- e. **마스터 프로세스 모니터링**: 스케줄러, API 서버 등 마스터 프로세스 모니터링 기능도 제공한다.
- f. **Prometheus 자체 모니터링**: Prometheus 자체의 스크래핑 엔드포인트 및 관련 정보를 확인할 수 있다.
- g. **사용자 정의 대시보드**: 필요에 따라 자체 대시보드를 생성할 수 있다.

3.2. Prometheus 자체 UI 활용

1. Prometheus UI 접근 방법

- a. 포트 포워딩을 통해 Prometheus UI에 접근할 수 있으며, Prometheus는 9090번 포트에서 실행된다.

2. Prometheus UI의 주요 기능

- a. **알림**: 알림을 표시하고, 실행 중인 메트릭 엔드포인트 등 구성을 확인할 수 있다.
- b. **PromQL 쿼리**: Prometheus PromQL 언어를 사용하여 다양한 정보를 쿼리할 수 있다.
- c. **기본 알림 규칙**: 기본 알림 규칙 파일에서 구성된 기본 알림들을 확인할 수 있다.
- d. **스크래핑 타겟**: 스크래핑 중인 각 타겟과 모니터링 중인 Pod 및 그 상태를 볼 수 있다.
- e. **규칙 구성**: 규칙 구성을 확인할 수 있다.
- f. Prometheus UI는 유용하고 흥미로운 정보를 많이 제공한다.

4. 요약 및 향후 계획

1. Helm을 이용한 Prometheus 배포 요약

- a. Helm을 사용하여 전체 Prometheus 모니터링 스택을 배포하는 과정은 매우 쉽고 간단했다.
- b. 단 하나의 명령으로 스택을 구성하는 모든 구성 요소를 생성할 수 있었다.

2. 향후 계획

- a. 다음 영상에서는 클러스터에서 실행 중인 데이터베이스 애플리케이션과 같은 서비스에 대한 추가 메트릭 엔드포인트를 구성하는 Prometheus 데모를 보여줄 예정이다.
- b. 클러스터에서 실행 중인 다양한 서비스와 자체 애플리케이션을 메트릭을 노출하여 모니터링하는 방법을 다룰 것이다.

5. Prometheus whit. PromQL